Danmarks Tekniske Universitet

MODELING CO2 CONCENTRATIONS USING HIDDEN MARKOV MODELS

2026

Special Course in Statistical Modelling

Mads Holme Håkonsson
s234936

Supervisor
Jan Klobbenborg Møller

31. maj 2026

AI Declaration

AI tools has been used for formatting LaTex figures, tables and equations and grammar checking in the report. No AI has been used for for writing the math or the academic content of the report or the code. No Agentic AI tools has been used for wrriting the src code or the report. Agentic AI tools has been used for writing some of testing files in the tests directory or refactoring code as renaming files, variables or functions.

Indhold

1 Introduction **3**

2 Software **4**
2.1 Class overview 4
2.2 Design Principles 4
2.3 Primary Libraries 4

3 Generel Theory **5**
3.1 Forward Algorithm 5
3.2 Forecast Pseudo Residuals 5
3.3 Test Statistics 6

4 Models & Results **7**
4.1 Ordinary HMM model 7
4.1.1 Theory 7
4.1.2 Results 8

5 Discussion **9**

6 Conclusion **10**

7 Future Work **11**

8 Appendix **13**
8.1 Source Code Repository 13
8.2 Source code & Envoriment 13
8.3 CO_2 Data 14
8.4 Class Diagrams 15
8.5 Class Design 17

1 Introduction

Hidden Markov Models (HMMs) are a powerful tool for modeling sequential data, where the underlying states of the system are not directly observable. They have been widely used in various fields such as speech recognition, natural language processing, and bioinformatics. [6].
The goal of this project is to inference if a person is in a room based on CO_2 data collected from a sensor in the room. To do this, we will use a Hidden Markov Model to model the relationship between the observed CO_2 levels and the hidden states of whether a person is in the room or not and if a window is open or not. When training the models, we will use `JAX` [2] and `EQUINOX` [4] for automatic differentiation and optimization, and we will use `Optax` [3] as our optimization library.
WRITE HERE THE RESULTS YOU FIND LATER ON.

This report is written for the purpose of given a concise outline of the projects achivements, design choices, model results and interpretaiton. The report is written for future students to be able to easily carry on the the project and extend it in some way.

2 Software

This section will will give a explanation of the classes implemented, the design choices made and the primarylibraries used. A repository link can be found in 8.1 and the envoriment details and short source code overview can be found in 8.2. Class diagrams can be found in 8.4.

2.1 Class overview

The source code is organized into 6 main components: the `HMM` class, the `Emissions` class, the `Transitions` class, the `HMMParams` class, the `Inference` class, and the `Solvers` class. All classes except `HMM` are abstract interfaces. The `HMM` class acts as the main entry point, providing a simple interface for training and inference by delegating to the other components. The key design principle is that concrete implementations of `Emissions` and `Transitions` are injected at runtime, allowing different HMM variants to be used interchangeably. A detailed explanation of each class and the motivation behind the design choices can be found in 8.5, and class diagrams are provided in 8.4.

2.2 Design Principles

An Object-Oriented Programming (OOP) design was chosen for this project to promote modularity, extensibility, and separation of concerns. The Dependency Inversion Principle (DIP) was central to the design of all core components. They are defined as abstract interfaces, so different implementations can be swapped in without affecting the rest of the system. This modularity is also motivated by the mathematical structure of HMMs themselves. The forward algorithm, and HMM inference in general, depends only on two quantities. The emission density $p(o_t \mid s_t)$ and the transition probability $p(s_t \mid s_{t-1})$. It is entirely agnostic to how those quantities are computed. By encapsulating each behind an abstract interface, different HMM variants can be used interchangeably without modifying the inference or optimization code, so modularity follows naturally from the structure of the model. This also makes it straightforward to extend the project by adding new emission distributions, transition models, or inference algorithms without modifying existing code.

2.3 Primary Libraries

The primary libraries used in this projects are `JAX` [2], `EQUINOX` [4], and `Optax` [3].
`JAX` was chosen over `NumPy` for its automatic differentiation capabilities and performance optimizations. However, `JAX` is designed for functional programming and does not natively support classes or object-oriented design. Therefore, `EQUINOX` was chosen as it's a library built on top of `JAX` that provides a simple and flexible interface for defining classes as pytrees. [1], `JAX` can then automatically differentiate trough the classes and the leaves of the pytree.
`Optax` was chosen as the optimization library for its seamless integration with `JAX` and `EQUINOX`, providing a wide range of optimization algorithms and utilities for training machine learning models. `Optax` was purely choosen as it's the default optimization library for `JAX` and there may be a better choice for this project, but it was not explored due to time constraints.

[1] A pytree is a nested data strucure in `JAX`:`https://docs.jax.dev/en/latest/pytrees.html`

3 Generel Theory

In this section, we will present the type of forward algorithm used to compute the likelihood and the pseudo residuals used to evaluate the model fit.

3.1 Forward Algorithm

The forward algorithm implemented is called recursive filtering [5] as it constructs the current statedistribution $p(X_t \mid Y_{1:t})$ recursively from the previous state distribution $p(X_t \mid Y_{1:t-1})$.

Algorithm 1 Recursive Filtering Forward Algorithm

Require: Observations $\mathbf{y} = (y_1, \ldots, y_T)$, covariates $\mathbf{x} = (x_1, \ldots, x_T)$, initial state distribution $\mathbf{u}_0 \in \mathbb{R}^{1\times N}$, HMM parameters θ

Ensure: Predicted distributions $\{\mathbf{u}_t\}$, filtered distributions $\{\mathbf{u}_{t|t}\}$, step likelihoods $\{f_t\}$

for $t = 1, \ldots, T$ **do**
  $\Gamma_t \leftarrow \textsc{TransitionMatrix}(\theta, t, y_t, x_t)$ ▷ $N \times N$ transition matrix
  $\mathbf{u}_{t|t-1} \leftarrow \mathbf{u}_{t-1|t-1} \cdot \Gamma_t$ ▷ Predicted state distribution, $1 \times N$
  $\mathbf{g}_t \leftarrow \textsc{Density}(\theta, t, y_t, x_t)$ ▷ Emission densities for all states, $1 \times N$
  $f_t \leftarrow \sum_{i=1}^{N} [\mathbf{u}_t \odot \mathbf{g}_t]_i$ ▷ Marginal likelihood at time t
  $f_t \leftarrow \max(f_t,\ 10^{-10})$ ▷ Clip to avoid division by zero
  $\mathbf{u}_{t|t} \leftarrow \dfrac{\mathbf{u}_t \odot \mathbf{g}_t}{f_t}$ ▷ Filtered state distribution (normalized)
end for
return $\{\mathbf{u}_t\},\ \{\mathbf{u}_{t|t}\},\ \{f_t\}$

where $\odot$ denotes element-wise multiplication of vectos.

The log-likelihood is obtained by summing the log of the step likelihoods:

$$\mathcal{L}(\theta) = \sum_{t=1}^{T} \log f_t \tag{1}$$

3.2 Forecast Pseudo Residuals

A method to access the goodness-of-fit of the HMM is to compute the pseudo residuals. For a introduction to pseudo residuals, see chapter 6.2 in [10].
The Forecast Pseudo Residuals used in this project compute the the inverse of the cummulative distribution function (CDF) of the one-step-ahead predictive distribution. If the model is is good, the pseudo residuals should be approximately standard normally distributed.

Briefly desribed, a forecast pseudo residual for a HMM is defined as

$$z_{t+1} = \Phi^{-1}(F_Y(y_{t+1} \mid Y_{1:t}))$$

where Φ^{-1} is the inverse CDF of the standard normal distribution and F_Y is the predictive CDF of Y_{t+1} given $Y_{1:t}$, defined as

$$F_Y(y_{t+1} \mid Y_{1:t}) = \sum_{i=1}^{N} u^i_{t+1|t} F_i(y_{t+1} \mid X = i)$$

where X is the hidden state.
Because z_{t+1} now should follow a standard normal distribution, we can use normality tests to assess the goodness-of-fit of the model.
In this project, we use the QQ-plot to visually assess the normality of the pseudo residuals.

3.3 Test Statistics

Other methods to assess the goodness of fit of the HMM model include the AIC [7] and BIC [8] test statistics. These are defined as

$$\text{AIC} = 2k - 2\mathcal{L}$$

$$\text{BIC} = k\log(n) - 2\mathcal{L}$$

where k is the number of parameters in the model, n is the number of observations and $\mathcal{L}$ is the log-likelihood of the model.
The AIC and BIC test statistics are used to compare different models, where a lower value initiates a better fit.
Hoever, a lot of the models implemented are nested models, so a Likelihood Ratio Test (LRT) [9] can also be used to compare the models. The LRT test statistic is defined as

$$\text{LRT} = 2(\mathcal{L}_1 - \mathcal{L}_0)$$

where $\mathcal{L}_1$ is the log-likelihood of the more complex model and $\mathcal{L}_0$ is the log-likelihood of the simpler model.
The LRT test statistic follows a chi-squared distribution with degrees of freedom equal to the difference in the number of parameters between the two models.

4 Models & Results

For generel theory on HMM's, see [10] and [5] chapter 11. All the models implemented are 4 state Gaussian HMMs (emission density is a Gaussian distribution) and we have kept the the mean value of state 1 fixed at 400 as this is the baseline CO_2 level in the room when no one is present and all windows are closed. This is also shown in the figure 1.

4.1 Ordinary HMM model

4.1.1 Theory

The ordinary HMM model is the simplest model implemented in this project and the one most commonly used in the literature. The transition probabilities are constant and do not depend on time, the observations or any covariates, and so does the emission density.
For this model, we want to estimate the parameters $\theta = \{\Gamma, \mu, \sigma^2\}$ where $\Gamma \in \mathbb{R}^{n\times n}$ is the transition matrix, $\mu \in \mathbb{R}^n$ is the mean vector of the Gaussian emission density and $\sigma^2 \in \mathbb{R}^n$ is the variance vector of the Gaussian emission density. Because the markov chain is finite, discrete and homogeneous, a unique strictly positive stationary distribution δ exists (chappter 1.3.2 [10]) and is given by

$$\delta = \mathbf{1}^T(I - \Gamma + E)^{-1}$$

where E is a matrix of ones.

To handle the natural bounds on the standard deviation parameters and transition proberbilities, the parameters were reparameterized as follows:

$$\sigma_i^2 = \exp(\eta_i)$$

For the transition matrix, only the $n(n-1)$ off-diagonal entries are parametrized; the diagonal is used as the reference category with log-odds fixed to zero. Concretely, the unconstrained parameters are stored as a matrix $\xi \in \mathbb{R}^{n\times(n-1)}$ that holds the off-diagonal logits. The transition matrix is then obtained by exponentiating these logits in the off-diagonal positions of an identity matrix (so the diagonal is $\exp(0) = 1$) and row-normalizing:

$$\Gamma_{ij} = \begin{cases} \dfrac{\exp(\xi_{ij})}{1 + \sum_{k\neq i}\exp(\xi_{ik})}, & j \neq i, \\ \dfrac{1}{1 + \sum_{k\neq i}\exp(\xi_{ik})}, & j = i. \end{cases}$$

This is equivalent to a row-wise softmax with the diagonal entry pinned as the reference, and guarantees $\Gamma_{ij} \geq 0$ and $\sum_j \Gamma_{ij} = 1$ for any $\xi \in \mathbb{R}^{n\times(n-1)}$. The stationary distribution δ is not parametrized directly; it is recomputed from Γ at every optimization step via $\delta = \mathbf{1}^T(I - \Gamma + E)^{-1}$, so reparametrizing Γ through ξ implicitly reparametrizes δ.

The unconstrained parameters $\eta \in \mathbb{R}^n$ and $\xi \in \mathbb{R}^{n\times(n-1)}$ are the quantities that are optimized during the fitting process.

The μ parameters are not bounded, but to avoid label switching for the gaussitan distrbitions, the mean parameters are ordered as $\mu_1 < \mu_2 < \mu_3 < \mu_4$ by the reparameterization

$$\mu_1 = \zeta_1$$

$$\mu_i = \zeta_1 + \sum_{k=2}^{i}\exp(\zeta_k)$$

where $\zeta \in \mathbb{R}^n$ is the unconstrained parameter vector that is optimized during the fitting process.

4.1.2 Results

5 Discussion

6 Conclusion

7 Future Work

Litteratur

[1] Astral Software Inc. uv: An ultra-fast python package installer and resolver. `https://docs.astral.sh/uv/`, 2024. Accessed: 2026-05-30.

[2] James Bradbury, Roy Frostig, Peter Hawkins, Matthew James Johnson, Yash Katariya, Chris Leary, Dougal Maclaurin, George Necula, Adam Paszke, Jake VanderPlas, Skye Wanderman-Milne, and Qiao Zhang. JAX: composable transformations of Python+NumPy programs, 2018.

[3] Matteo Hessel, David Budden, Fabio Viola, Mihaela Rosca, Eren Sezener, and Tom Hennigan. Optax: composable gradient transformation and optimisation, in jax!, 2020.

[4] Patrick Kidger and Cristian Garcia. Equinox: neural networks in JAX via callable PyTrees and filtered transformations. *Differentiable Programming workshop at Neural Information Processing Systems 2021*, 2021.

[5] Jan Kloppenborg Møller, Marcel Schweiker, Rune Korsholm Andersen, Burak Gunay, Selin Yilmaz, Verena Marie Barthelmes, and Henrik Madsen. *Statistical Modelling of Occupant Behaviour*. A Chapman & Hall Book. Chapman and Hall/CRC, 1st edition, 2024.

[6] Ultralytics LLC. Hidden markov model (hmm) - real-world applications. `https://www.ultralytics.com/glossary/hidden-markov-model-hmm#real-world-applications`, 2024. Accessed: 2026-05-30.

[7] Wikipedia contributors. Akaike information criterion — Wikipedia, the free encyclopedia, 2026. [Online; accessed 31-May-2026].

[8] Wikipedia contributors. Bayesian information criterion — Wikipedia, the free encyclopedia, 2026. [Online; accessed 31-May-2026].

[9] Wikipedia contributors. Likelihood-ratio test — Wikipedia, the free encyclopedia, 2026. [Online; accessed 31-May-2026].

[10] Walter Zucchini, Iain L. MacDonald, and Roland Langrock. *Hidden Markov Models for Time Series: An Introduction Using R*. Number 150 in Monographs on Statistics and Applied Probability. CRC Press, second edition, 2016.

8 Appendix

8.1 Source Code Repository

The repository is hosted on GitHub and the source code can be found there with the data as well: `https://github.com/Maccen26/hmm-project`

8.2 Source code & Envoriment

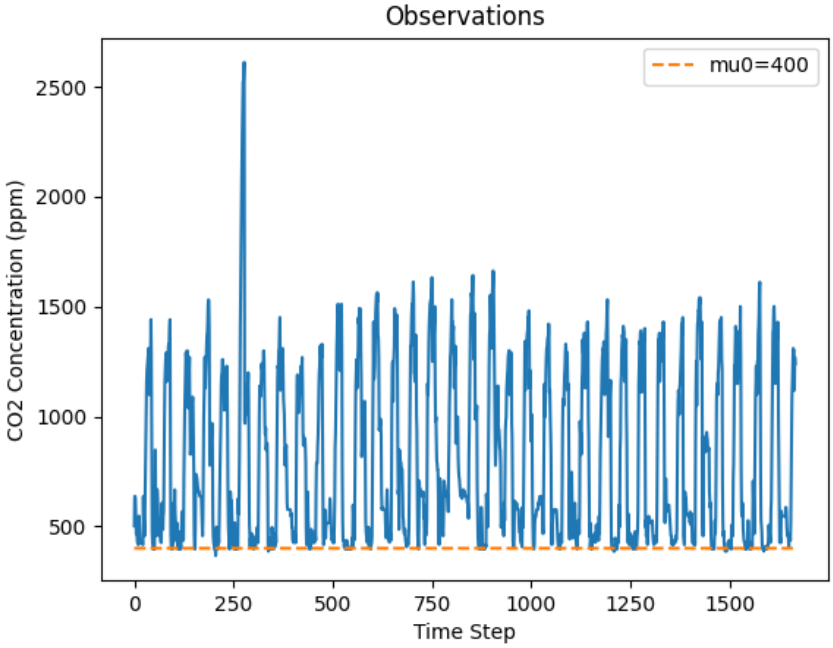
This code was written with Python (version 3.11) and `uv` (version 0.9.7) [1] was used as a package manager. The libraries used are listed in Table 1.

Library	**Version**
`dotenv`	$\geq 0.9.9$
`equinox`	$\geq 0.13.7$
`jax`	$\geq 0.9.0.1$
`matplotlib`	$\geq 3.10.8$
`numpy`	$\geq 1.23.5, < 2.3$
`optax`	$\geq 0.2.8$
`pandas`	$\geq 3.0.0$
`seaborn`	$\geq 0.13.2$
`statsmodels`	$\geq 0.14.6$

Tabel 1: Python libraries used in this project.

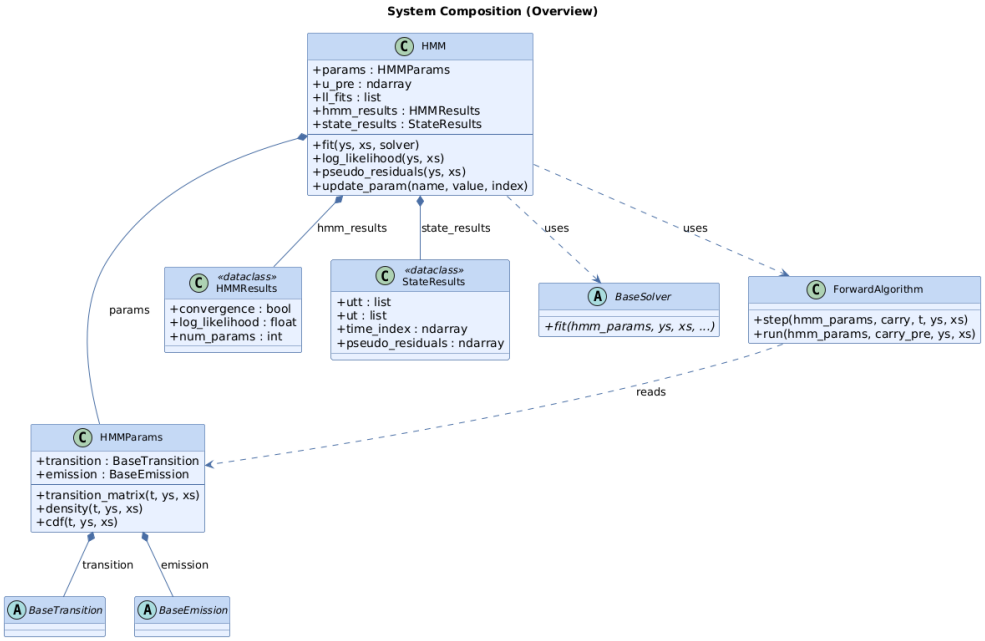
The source code is organized in a way such that the directory `src` contains the source code for the project, the directory `data` contains the data used for training and testing the model, the directory `tests` contains tests for the source code and the directory `drivers` contains scripts for running the model and generating the results used in the report.

8.3 CO_2 Data

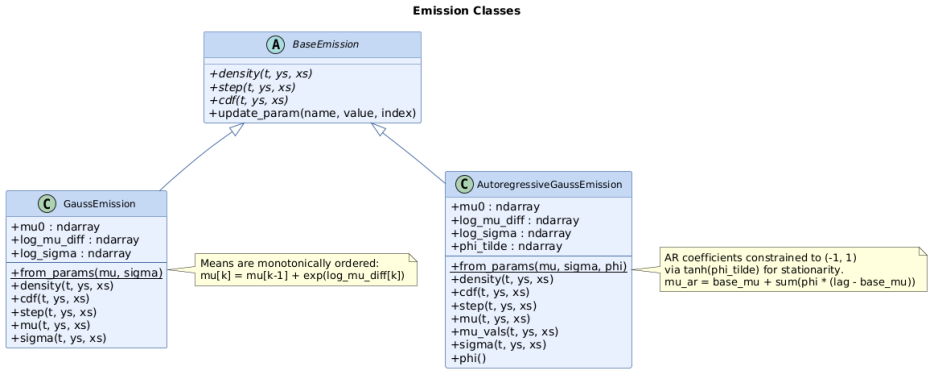


Figur 1: Observed CO_2 concentration (ppm) over time.

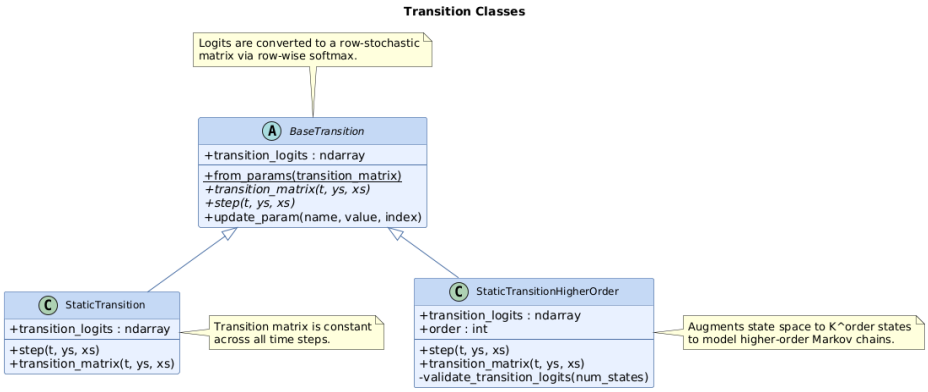
8.4 Class Diagrams



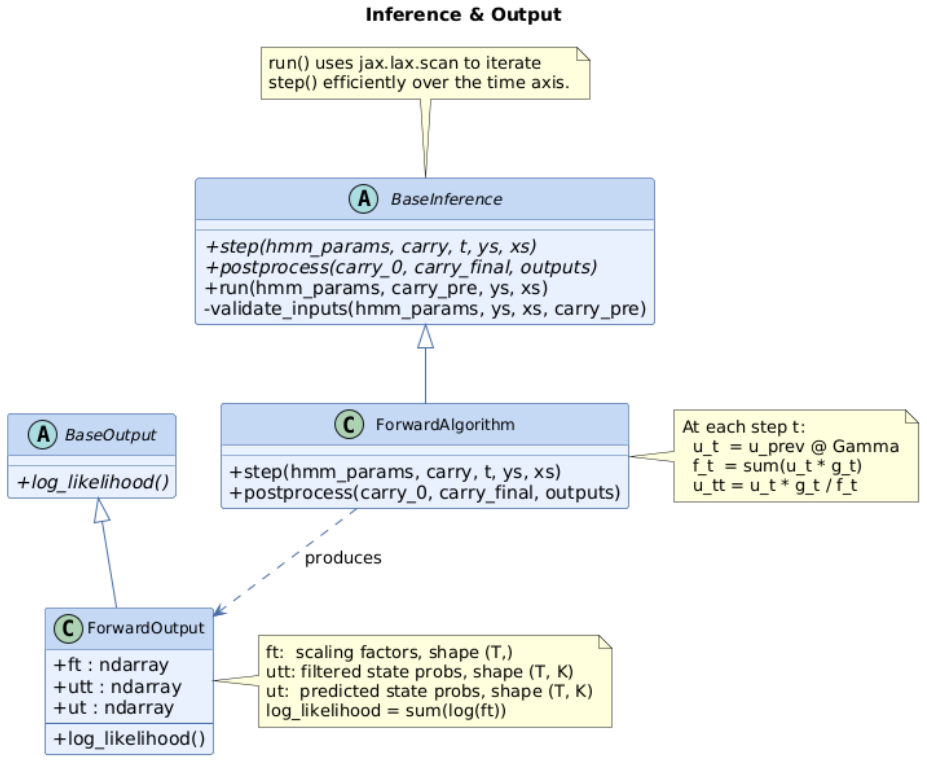
Figur 2: System composition overview. The user-facing `HMM` class owns an `HMMParams` instance composed of a `BaseTransition` and a `BaseEmission`. Fitting is delegated to a `BaseSolver` subclass and state inference is handled by `ForwardAlgorithm`, producing `HMMResults` and `StateResults`.



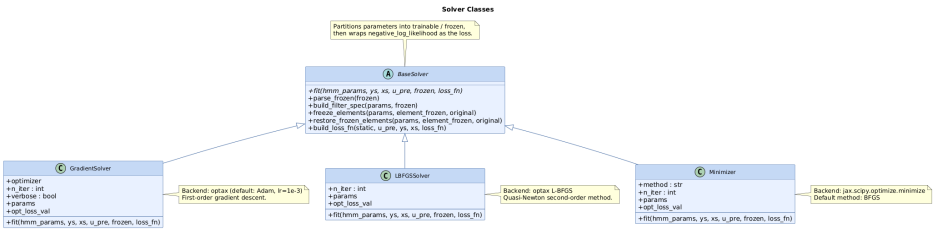
Figur 3: Emission class hierarchy. `GaussEmission` models state-dependent Gaussian distributions with monotonically ordered means enforced via cumulative exponentiation of `log_mu_diff`. `AutoregressiveGaussEmission` extends this with AR(p) dynamics, where coefficients are stored unconstrained (`phi_tilde`) and mapped to $(-1, 1)$ via tanh to guarantee stationarity.



Figur 4: Transition class hierarchy. Both implementations store parameters as logits and convert them to a row-stochastic matrix via row-wise softmax. `StaticTransition` uses a constant $K \times K$ matrix, while `StaticTransitionHigherOrder` augments the state space to K^{order} states to represent a higher-order Markov chain.



Figur 5: Inference and output class hierarchy. `BaseInference.run()` iterates `step()` across the time axis using `jax.lax.scan`. `ForwardAlgorithm` implements the forward filtering pass and produces a `ForwardOutput` storing the scaling factors f_t, filtered state probabilities $u_{t|t}$, and predicted state probabilities u_t. The log-likelihood is computed as $\sum_t \log f_t$.



Figur 6: Solver class hierarchy. `BaseSolver` partitions parameters into trainable and frozen subsets and wraps the negative log-likelihood as the loss function. The three concrete solvers share the same interface but differ in their optimization backend: `GradientSolver` uses first-order gradient descent via `optax`, `LBFGSSolver` uses quasi-Newton L-BFGS via `optax`, and `Minimizer` wraps `jax.scipy.optimize.minimize`.

8.5 Class Design

The `HMM` class (2) is the main entry point of the system. Its responsibility is to act as a wrapper for the other components and to provide a simple interface for training and inference. It is dependent on `Emissions`, `Transitions`, `HMMParams`, and `Solvers`, which are composed together to form a complete HMM.

The `Emissions` (3) and `Transitions` (4) classes are abstract interfaces whose concrete implementations must be provided at runtime. This design choice is motivated by the structure of HMM variants: the differences between models in this project lie in how the density of an observation given a state is computed (emissions) and how the transition probabilities between states are computed (transitions). By separating these concerns into abstract interfaces, different HMM models can be composed without changing the rest of the system.

The `HMMParams` class acts as a container for the model parameters and as a wrapper for the methods implemented in the `Emissions` and `Transitions` classes. This is required by how `JAX` and `Equinox` work: parameters must be stored as pytrees to support automatic differentiation.

The `Inference` class (5) is an abstract interface responsible for implementing inference algorithms such as the forward algorithm, the backward algorithm, and the Viterbi algorithm. In this project, only the forward algorithm is implemented, but the design allows easy extension to other inference algorithms without modifying existing code.

The `Solvers` class (6) implements the optimization algorithms and depends on a loss function that takes the `HMMParams` and `Inference` classes as arguments. This dependency injection of the loss function decouples the optimizer from the objective, making it straightforward to change the loss function without altering the optimization algorithm.